DSA ASSIGNMENT

**1.Describe the greedy algorithm with example.**

**Greedy Algorithm:**

A greedy algorithm is a problem-solving approach that makes a series of decisions, each time choosing the best option available at that moment (locally optimal) with the hope that these local decisions will lead to the global optimum. This approach works well for optimization problems where making a series of locally optimal choices results in the best overall solution.

**Key Characteristics:**

1. Greedy choice property: At every step, choose the best possible option.

2. Optimal substructure: A problem has an optimal solution if its subproblems also have optimal solutions.

**Steps of a Greedy Algorithm:**

1. Initialization: Start with an initial solution or state (often empty or a minimal configuration).

2. Selection: At each step, choose the option that seems best (locally optimal choice).

3. Feasibility Check: Ensure that the choice satisfies the problem's constraints.

4. Incorporation: Include the selected choice in the current solution.

5. Repeat: Continue until no more choices are left, or a required condition is met.

6. Solution: The final result is the accumulated choices.

**Example: Activity Selection Problem**

Given n activities with their start and end times, the objective is to select the maximum number of activities that do not overlap.

Activities:

1. Activity A1: (1, 4)

2. Activity A2: (3, 5)

3. Activity A3: (0, 6)

4. Activity A4: (5, 7)

5. Activity A5: (8, 9)

6. Activity A6: (5, 9)

The first number is the start time and the second is the end time of the activities.

**Steps to Solve using Greedy Approach:**

1. Sort the activities based on their end times (in ascending order):

A1: (1, 4)

A2: (3, 5)

A4: (5, 7)

A6: (5, 9)

A5: (8, 9)

A3: (0, 6)

2. Select the first activity (A1) since it has the earliest end time (1, 4).

3. For each subsequent activity, select it only if its start time is greater than or equal to the end time of the previously selected activity.

A2: (3, 5) – Overlaps with A1 (not selected)

A4: (5, 7) – Does not overlap with A1 (selected)

A5: (8, 9) – Does not overlap with A4 (selected)

4. Solution: The selected activities are:

A1: (1, 4)

A4: (5, 7)

A5: (8, 9)

**Advantages and Disadvantages of Greedy Algorithms:**

Advantages:

Simple to implement.

Efficient for many problems (e.g., activity selection, Huffman coding).

Disadvantages:

May not always yield the optimal solution.

Requires problem-specific insight to determine if the greedy approach works.

**2. For all algorithms what is the time complexity, space complexity and data structure in use.**

**1. Coin Change Problem**

• Objective: Minimize the number of coins needed to make a specific amount.

**Time Complexity:**

• Sorted Coin Denominations: O (1)O(1)O(1) per coin selection (since we use the largest denomination).

• Unsorted Denominations: O (n log⁡ n) O (n \log n) O (n log n), where n is the number of available coin denominations (if sorting is required).

**Space Complexity:**

• O (1): Only a fixed amount of space is needed regardless of input size (for storing remaining amount and coin count).

**Data Structure:**

• Array or List: To store the coin denominations.

**2. Activity Selection Problem**

• Objective: Select the maximum number of non-overlapping activities.

**Time Complexity:**

Sorting activities by finish time: O (n log⁡ n) O (n \log n) O (n log n), where n is the number of activities.

• Selecting activities after sorting: O(n)O(n)O(n) for iterating through the list.

**Space Complexity:**

• O(n): For storing the list of activities.

**Data Structure:**

• Array or List: To store the activities and their start and finish times.

**3. Fractional Knapsack Problem**

• Objective: Maximize the total value of items in the knapsack by allowing fractions of items.

**Time Complexity:**

• Sorting items by value-to-weight ratio: O (n log⁡n) O (n \log n) O (n log n).

• Filling the knapsack: O(n)O(n)O(n) for iterating through the sorted items.

**Space Complexity:**

• O(n): For storing the items values, weights, and value-to-weight ratios.

**Data Structure:**

• Array or List: To store item weights, values, and their ratios.

• Sorting algorithms like Merge Sort or Quick Sort may be used to sort the items by ratio.

**4. Huffman Coding**

• Objective: Create an optimal prefix code based on the frequencies of characters.

**Time Complexity:**

• Building the priority queue (min-heap): O (n log⁡ n) O (n \log n) O (n log n), where

n is the number of characters.

• Building the Huffman tree: O (n log n) O (n \log n) O (n log n).

**Space Complexity:**

O(n): For storing the frequency table and the Huffman tree.

**Data Structure:**

• Priority Queue (Min-Heap): To repeatedly pick the two smallest frequencies nodes.

• Binary Tree: To construct the Huffman coding tree.

**5. Prim's Algorithm (for Minimum Spanning Tree)**

• Objective: Find the Minimum Spanning Tree of a graph.

**Time Complexity:**

• Using a priority queue (min-heap): O((V+E) log V) O ((V + E) \log V) O((V+E) log V), where VVV is the number of vertices and EEE is the number of edges.

• Without priority queue (adjacency matrix): O(V2) O(V^2) O(V2).

**Space Complexity:**

• O (V + E): To store the graph and the Minimum Spanning Tree.

**Data Structure:**

• Priority Queue (Min-Heap): To select the next minimum weight edge.

• Adjacency List: To store the graph (if using a more efficient implementation).

• Array: To track visited vertices and their corresponding costs.

**6. Kruskal's Algorithm (for Minimum Spanning Tree)**

• Objective: Find the Minimum Spanning Tree of a graph by sorting edges.

**Time Complexity:**

• Sorting edges: O (E log E) O (E \log E) O (E log E), where EEE is the number of edges.

• Union-Find operations: O (E log V) O (E \log V) O (E log V), using a Union-Find/Disjoint Set structure, where VVV is the number of vertices.

**Space Complexity:**

• O (V + E): To store the graph and the disjoint-set data structure.

**Data Structure:**

• Union-Find/Disjoint Set: For cycle detection when adding edges to the MST.

• Array: To store the parent and rank for the Union-Find operations.

• Edge List: To store and sort the graph's edges.

**7. Dijkstra's Algorithm (for Shortest Path)**

• Objective: Find the shortest path from a source vertex to all other vertices in a weighted graph.

**Time Complexity:**

• Using a priority queue (min-heap): O((V+E) log)) O ((V + E) \log V)O((V+E) log V), where VVV is the number of vertices and EEE is the number of edges.

• Without a priority queue (using arrays): O(V2) O(V^2) O(V2).

**Space Complexity:**

• O (V + E): To store the graph and the shortest paths.

**Data Structure:**

• Priority Queue (Min-Heap): To extract the next vertex with the minimum distance.

• Adjacency List: To represent the graph.

• Array: To store distances from the source vertex and to track visited vertices.

**3. What is Dynamic Programming? What are the types? Different algorithms.**

Dynamic Programming (DP) is an optimization technique used to solve complex problems by breaking them down into overlapping subproblems. It stores the results of these subproblems to avoid redundant computations, which makes it particularly effective for problems with overlapping subproblems and optimal substructure properties.

Overlapping Subproblems: The problem can be divided into subproblems that are reused several times.

Optimal Substructure: The solution to a problem can be constructed efficiently using solutions to its subproblems.

**Types of Dynamic Programming:**

**1. Top-Down Approach (Memoization)**

Start solving the problem from the top, i.e., the main problem, and recursively solve smaller subproblems.

Store the results of these subproblems in a table (cache) to avoid redundant calculations.

Example: Fibonacci sequence using recursion + memoization.

**2. Bottom-Up Approach (Tabulation)**

Solve all the subproblems first and use their results to build up the solution to the main problem.

Uses a table to store intermediate results iteratively, rather than using recursion.

Example: Fibonacci sequence using tabulation.

**Common Dynamic Programming Algorithms:**

1. Fibonacci Sequence – Calculate the n-th Fibonacci number efficiently using DP.

Type: Both Top-Down and Bottom-Up.

2. Longest Common Subsequence (LCS) – Find the longest subsequence common to two strings.

Type: Bottom-Up (Table-based).

3. Knapsack Problem – Find the maximum value that can be obtained from a given set of items with weight constraints.

Type: Bottom-Up (Tabulation).

4. Coin Change Problem – Find the minimum number of coins needed to make a given amount.

Type: Bottom-Up.

5. Matrix Chain Multiplication – Minimize the number of scalar multiplications in matrix multiplication.

Type: Bottom-Up.

6. Shortest Path in Graphs (Bellman-Ford Algorithm) – Find the shortest path from a source to all vertices.

Type: Bottom-Up.

7. Edit Distance (Levenshtein Distance) – Calculate the minimum number of operations needed to convert one string into another.

Type: Bottom-Up.

8. Rod Cutting Problem – Maximize the profit by cutting a rod of given length into pieces.

Type: Bottom-Up.

**4.Types of Dynamic Programming, time complexity, space complexity and data structures use.**

**1. Fibonacci Sequence**

• Time Complexity: O(n)O(n)O(n) (with memoization or tabulation)

• Space Complexity: O(n)O(n)O(n) (for memoization) or O (1)O(1)O(1) (for

iterative approach)

• Data Structure: Array (for storing Fibonacci values) or variables (for

iterative).

**2. 0/1 Knapsack Problem**

• Time Complexity: O (n W) O (n W) O (n W) (where n is the number of items and WWW is the maximum capacity)

• Space Complexity: O (n W) O (n W) O (n W) (using a 2D table) or O(W)O(W)O(W) (using a 1D table for space optimization)

• Data Structure: 2D array or 1D array.

**3. Longest Common Subsequence (LCS)**

• Time Complexity: O (m n) O (m n) O (m n) (where mmm and n are the lengths of the two strings)

• Space Complexity: O (m n) O (m n) O (m n) (for a 2D table)

• Data Structure: 2D array.

**4. Longest Increasing Subsequence (LIS)**

• Time Complexity: O(n2) O(n^2) O(n2) (using dynamic programming) or O (n log ⁡n) O (n \log n) O (n log n) (using binary search)

• Space Complexity: O(n)O(n)O(n)

• Data Structure: 1D array.

**5. Matrix Chain Multiplication**

• Time Complexity: O(n3) O(n^3) O(n3) (where n is the number of matrices)

• Space Complexity: O(n2) O(n^2) O(n2) (for storing minimum costs)

• Data Structure: 2D array.

**6. Edit Distance (Levenshtein Distance)**

• Time Complexity: O (m n) O (m n) O (m n) (where mmm and n are the lengths of the two strings)

• Space Complexity: O (m n) O (m n) O (m n) (for a 2D table) or O(n)O(n)O(n) (using only two rows for space optimization)

• Data Structure: 2D array or 1D array.

**7. Coin Change Problem**

• Time Complexity: O (nm) O (n \times m) O (nm) (where n is the amount and mmm is the number of coin denominations)

• Space Complexity: O(n)O(n)O(n) (for storing ways to make change)

• Data Structure: 1D array.

**8. Rod Cutting Problem**

• Time Complexity: O(n2) O(n^2) O(n2)

• Space Complexity: O(n)O(n)O(n)

• Data Structure: 1D array.

**9. Travelling Salesman Problem (TSP)**

• Time Complexity: O (n2·2n) O (n^2 \c dot 2^n) O (n2·2n) (using dynamic programming)

• Space Complexity: O (n·2n) O (n \c dot 2^n) O (n·2n)

• Data Structure: 2D array or bitmask.

**10. Bellman-Ford Algorithm**

• Time Complexity: O(VE)O(VE)O(VE) (where VVV is the number of vertices and EEE is the number of edges)

• Space Complexity: O(V)O(V)O(V)

• Data Structure: Array (for distances) or adjacency list/array.

**11. Dijkstra's Algorithm**

• Time Complexity: O((V+E) log V) O ((V + E) \log V) O((V+E) log V) (using a priority queue)

• Space Complexity: O(V)O(V)O(V)

• Data Structure: Priority queue (min-heap) and array (for distances).

**12. Floyd-Warshall Algorithm**

• Time Complexity: O(V3) O(V^3) O(V3)

• Space Complexity: O(V2) O(V^2) O(V2)

• Data Structure: 2D array (for distances).

**13. Palindromic Subsequence**

• Time Complexity: O(n2) O(n^2) O(n2) (where n is the length of the string)

• Space Complexity: O(n2) O(n^2) O(n2) (for a 2D table)

• Data Structure: 2D array.

**14. Wildcard Matching**

• Time Complexity: O (m n) O (m n) O (m n) (where mmm is the length of the string and n is the length of the pattern)

• Space Complexity: O (m n) O (m n) O (m n) (for a 2D table) or O(n)O(n)O(n) (using a single row)

• Data Structure: 2D array or 1D array.

**15. Regular Expression Matching**

• Time Complexity: O (m n) O (m n) O (m n) (where mmm is the length of the string and n is the length of the pattern)

• Space Complexity: O (m n) O (m n) O (m n) (for a 2D table) or O(n)O(n)O(n) (using a single row)

• Data Structure: 2D array or 1D array.